

LAPORAN TUGAS

**WEB SERVER
(TIF 330805)**

MINGGU KE 5



ACARA 10

DIBUAT OLEH:

NAMA: CINTYA AZWA SAFRINA R.H

NIM: E41250631

GOL: A

**PROGRAM STUDI DI LUAR KAMPUS UTAMA (PSDKU) POLITEKNIK NEGERI
JEMBER
TEKNIK INFORMATIKA KAB. NGANJUK JL. JENDRAL GATOT SUBROTO NO
106, KAUMAN, KECAMATAN NGANJUK, KABUPATEN NGANJUK
JURUSAN TEKNOLOGI INFORMASI POLITEKNIK NEGERI JEMBER TAHUN
2026**

PENDAHULUAN

Pada Acara 9, MahasiswaController sudah memakai Repository dan Dependency Injection. Namun, di dalam Controller masih terdapat kode yang berulang di banyak method, yaitu memuat file View dengan `require_once` dan mengarahkan pengguna dengan `header("Location: ...")` diikuti `exit`. Jika Controller lain (Prodi, Mata Kuliah, dan seterusnya) juga menulis kode yang sama, akan terjadi duplikasi.

Praktikum ini menerapkan **Inheritance (pewarisan)** pada OOP. Inheritance memungkinkan sebuah class (class anak) mewarisi properti dan method dari class lain (class induk) dengan kata kunci `extends`, sehingga kode yang sama cukup ditulis satu kali di class induk. Method yang ditandai `protected` dapat dipakai oleh class anak tetapi tidak dapat dipanggil dari luar class.

Dibuat sebuah class induk Controller (BaseController) pada folder Core yang berisi dua method bersama: `view()` untuk memuat View beserta datanya, dan `redirect()` untuk mengarahkan pengguna ke URL lain. MahasiswaController kemudian mewarisi class tersebut sehingga otomatis memiliki kedua method itu. Selain itu, seluruh query SQL tetap berada di Repository sehingga Controller hanya mengatur alur request dan response.

IMPLEMENTASI

1. Menyiapkan folder Acara_10

Project dari Acara 9 digunakan sebagai dasar pengerjaan pada folder Acara_10. Perubahan yang dilakukan adalah penambahan file app/Core/Controller.php dan perubahan pada MahasiswaController.

```

app
├── Controllers  MahasiswaController.php (diubah: extends Controller)
├── Core         Controller.php (BARU), Database.php
├── Entities     Mahasiswa.php
├── Middleware
├── Models
├── Repositories MahasiswaRepository.php
└── Views
  
```

2. Membuat class induk Controller

File app/Core/Controller.php dibuat. Method view() memakai extract(\$data) agar isi array data berubah menjadi variabel yang langsung bisa dipakai di file View. Method redirect() mengirim header Location lalu menghentikan eksekusi dengan exit. Keduanya protected sehingga hanya dapat dipakai oleh class anak.

```

<?php
class Controller
{
    // Memuat file View, dengan $data diubah menjadi variabel-variabel
    // yang bisa langsung dipakai di dalam file View tersebut.
    protected function view(string $view, array $data = []): void
    {
        extract($data);
        require __DIR__ . '/../Views/' . $view . '.php';
    }

    // Mengarahkan pengguna ke URL lain, lalu menghentikan eksekusi script.
    protected function redirect(string $url): void
    {
        header("Location: {$url}");
        exit;
    }
}
  
```

3. Mewarisi class Controller pada MahasiswaController

MahasiswaController diubah menjadi class MahasiswaController extends Controller. Perubahan pemanggilannya:

Aspek	Acara 9 (sebelum)	Acara 10 (sesudah)
Deklarasi class	class MahasiswaController	class MahasiswaController extends Controller
Memuat View	require_once __DIR__ . '../Views/mahasiswa/index.php';	\$this->view('mahasiswa/index', ['daftarMahasiswa' => \$daftarMahasiswa]);
Redirect	header("Location: ..."); exit;	\$this->redirect("...");

Berikut isi lengkap MahasiswaController setelah perubahan.

```
<?php
require_once __DIR__ . '/../Core/Controller.php';
require_once __DIR__ . '/../Repositories/MahasiswaRepository.php';
require_once __DIR__ . '/../Entities/Mahasiswa.php';
require_once __DIR__ . '/../Models/ProdiModel.php';

// MahasiswaController mewarisi (extends) Controller, sehingga otomatis
// punya method view() dan redirect() tanpa perlu menulis ulang.
class MahasiswaController extends Controller
{
    private MahasiswaRepository $repo;

    // Constructor Dependency Injection: Controller menerima Repository dari luar.
    // Seluruh query SQL berada di MahasiswaRepository, tidak ada SQL di sini.
    public function __construct(MahasiswaRepository $repo)
    {
        $this->repo = $repo;
    }

    private function base(): string
    {
        return str_replace('\\', '/', dirname($_SERVER['SCRIPT_NAME']));
    }

    public function index(): void
    {
        $keyword = trim($_GET['q'] ?? '');
        $daftarMahasiswa = $this->repo->all($keyword !== '' ? $keyword : null);

        // Memakai method view() yang diwariskan dari Controller (BaseController)
        $this->view('mahasiswa/index', ['daftarMahasiswa' => $daftarMahasiswa]);
    }

    public function create(): void
    {
        $prodiModel = new ProdiModel();
        $daftarProdi = $prodiModel->all();

        $this->view('mahasiswa/create', ['daftarProdi' => $daftarProdi]);
    }

    public function store(): void
    {
        try {
            // Getter/setter pada class Mahasiswa otomatis memvalidasi input di sini
            $mahasiswa = new Mahasiswa(
                trim($_POST['nim'] ?? ''),
                trim($_POST['nama'] ?? ''),
                trim($_POST['email'] ?? ''),
                (int) ($_POST['prodi_id'] ?? 0),
                (int) ($_POST['angkatan'] ?? date('Y')),
                $_POST['status'] ?? 'aktif'
            );

            // Controller tidak menulis SQL -- cukup memanggil method Repository
            $this->repo->create($mahasiswa);

            // Memakai method redirect() yang diwariskan dari Controller
            $this->redirect("{${this->base()}}/mahasiswa");
        } catch (InvalidArgumentException $e) {
            $pesan = urlencode($e->getMessage());
            $this->redirect("{${this->base()}}/mahasiswa/create?error={$pesan}");
        }
    }
}
```

```

    }

    public function edit(int $id): void
    {
        $mahasiswa = $this->repo->find($id);

        if ($mahasiswa === null) {
            http_response_code(404);
            echo "404 - Mahasiswa tidak ditemukan";
            return;
        }

        $prodiModel = new ProdiModel();
        $daftarProdi = $prodiModel->all();

        $this->view('mahasiswa/edit', [
            'mahasiswa' => $mahasiswa,
            'daftarProdi' => $daftarProdi,
        ]);
    }

    public function update(int $id): void
    {
        try {
            $mahasiswa = new Mahasiswa(
                trim($_POST['nim'] ?? ''),
                trim($_POST['nama'] ?? ''),
                trim($_POST['email'] ?? ''),
                (int) ($_POST['prodi_id'] ?? 0),
                (int) ($_POST['angkatan'] ?? date('Y')),
                $_POST['status'] ?? 'aktif',
                $id
            );

            $this->repo->update($id, $mahasiswa);
            $this->redirect("{$_this->base()}/mahasiswa");
        } catch (InvalidArgumentException $e) {
            $pesan = urlencode($e->getMessage());
            $this->redirect("{$_this->base()}/mahasiswa/{$_id}/edit?error={$pesan}");
        }
    }

    public function destroy(int $id): void
    {
        $this->repo->delete($id);
        $this->redirect("{$_this->base()}/mahasiswa");
    }
}

```

4. Memastikan Controller dan routes tetap berfungsi

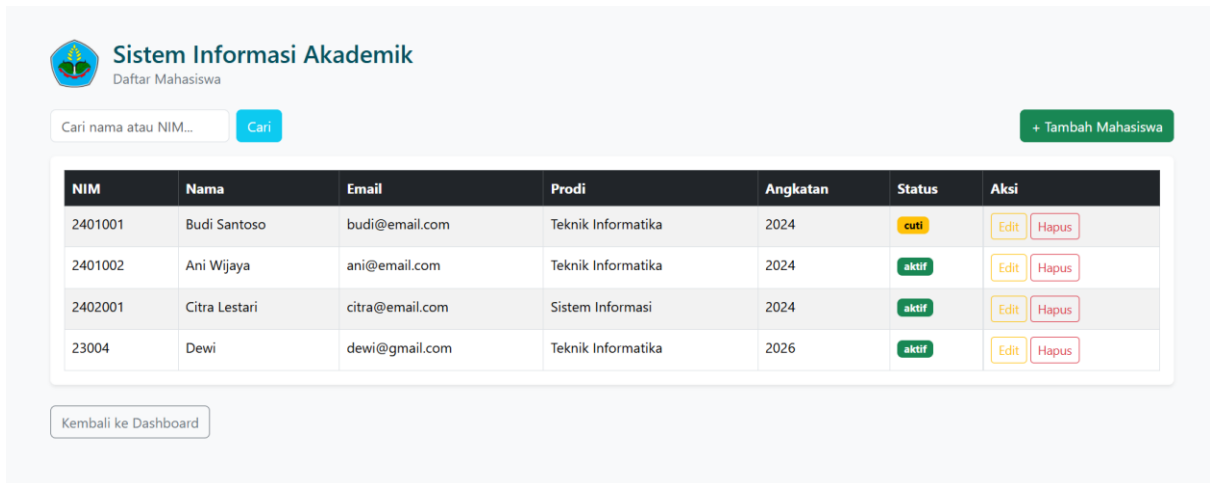
File routes/web.php tidak perlu diubah karena constructor MahasiswaController tetap menerima MahasiswaRepository. Constructor class anak dipakai langsung, sedangkan method view() dan redirect() berasal dari class induk.

HASIL PENGUJIAN

Karena refactoring ini hanya mengubah struktur kode, seluruh halaman diuji ulang untuk memastikan tampilan dan fungsi tetap sama seperti sebelumnya.

No	Skenario	Hasil yang Diharapkan	Hasil
1	Membuka halaman daftar mahasiswa	Halaman tampil melalui \$this->view()	Sesuai
2	Menambah mahasiswa dengan data valid	Data tersimpan, redirect ke daftar melalui \$this->redirect()	Sesuai
3	Menambah mahasiswa dengan NIM berisi huruf	Redirect ke form dengan pesan error	Sesuai
4	Mengubah dan menghapus data	Data berubah atau hilang, redirect ke daftar	Sesuai

a. Halaman daftar mahasiswa



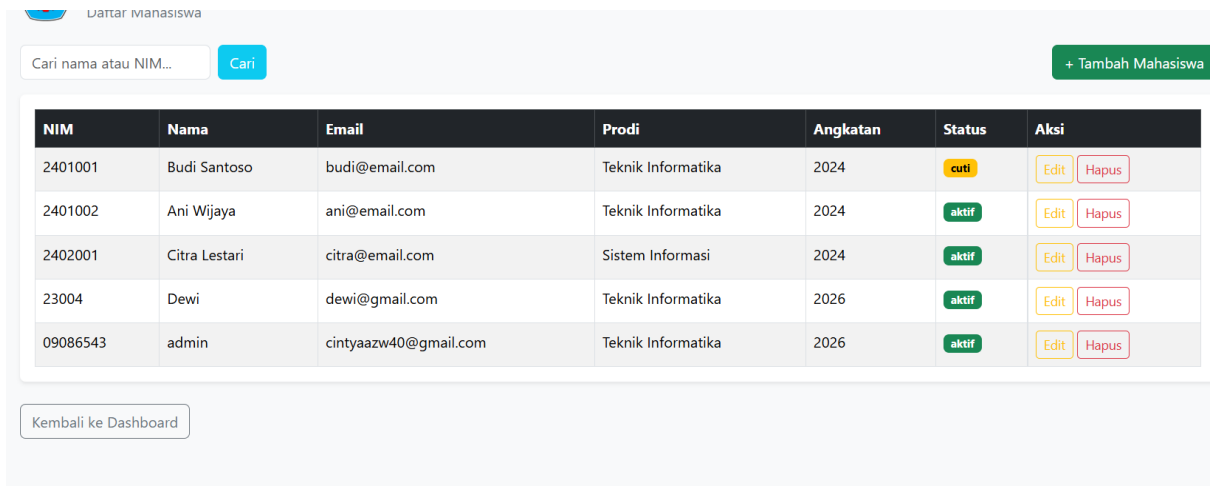
Sistem Informasi Akademik
Daftar Mahasiswa

Cari nama atau NIM... Cari + Tambah Mahasiswa

NIM	Nama	Email	Prodi	Angkatan	Status	Aksi
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	cuti	Edit Hapus
2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	aktif	Edit Hapus
2402001	Citra Lestari	citra@email.com	Sistem Informasi	2024	aktif	Edit Hapus
23004	Dewi	dewi@gmail.com	Teknik Informatika	2026	aktif	Edit Hapus

Kembali ke Dashboard

b. Tambah data valid



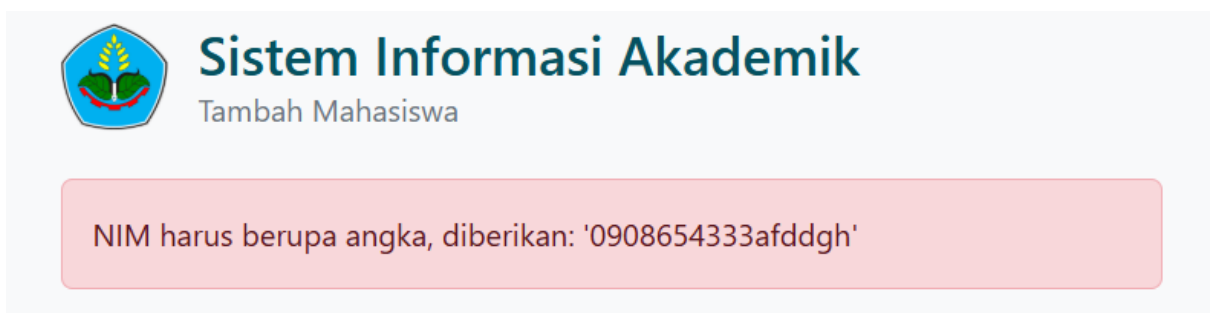
Sistem Informasi Akademik
Daftar Mahasiswa

Cari nama atau NIM... Cari + Tambah Mahasiswa

NIM	Nama	Email	Prodi	Angkatan	Status	Aksi
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	cuti	Edit Hapus
2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	aktif	Edit Hapus
2402001	Citra Lestari	citra@email.com	Sistem Informasi	2024	aktif	Edit Hapus
23004	Dewi	dewi@gmail.com	Teknik Informatika	2026	aktif	Edit Hapus
09086543	admin	cintyaazw40@gmail.com	Teknik Informatika	2026	aktif	Edit Hapus

Kembali ke Dashboard

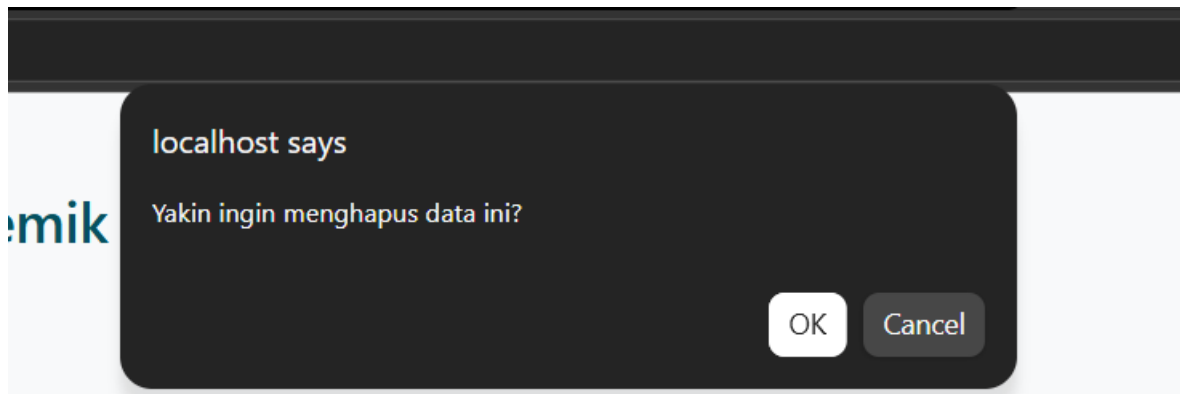
c. Validasi data tidak valid



Sistem Informasi Akademik
Tambah Mahasiswa

NIM harus berupa angka, diberikan: '0908654333afddgh'

d. Ubah dan hapus data



NIM	Nama	Email	Prodi	Angkatan	Status	Aksi
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	lulus	Edit Hapus
2401002	Ani Wiliava	ani@email.com	Teknik Informatika	2024	aktif	Edit Hapus

ANALISIS

Dengan extends Controller, MahasiswaController otomatis memiliki method view() dan redirect() tanpa menulis ulang. Sebelumnya, setiap method yang menampilkan halaman harus menulis require_once ke file View, dan setiap method yang berpindah halaman harus menulis header() dan exit. Kini cukup satu baris pemanggilan method, sehingga kode Controller lebih ringkas dan mudah dibaca.

Method view() memakai extract() untuk mengubah array data menjadi variabel di View. Contohnya ['daftarMahasiswa' => \$daftarMahasiswa] menjadi variabel \$daftarMahasiswa di file View. Method redirect() menjamin exit selalu dipanggil setelah header dikirim, sehingga tidak ada kode setelah redirect yang ikut berjalan tanpa sengaja.

Visibilitas protected pada kedua method di class induk membatasi pemakaian hanya untuk class turunan. Method tersebut tidak dapat dipanggil dari luar, sehingga encapsulation tetap terjaga.

Keuntungan utama inheritance terlihat ketika Controller lain, seperti ProdiController dan MatakuliahController, nanti ikut mewarisi Controller. Perubahan cara memuat View atau melakukan redirect cukup dilakukan di satu tempat. Hal ini sejalan dengan prinsip DRY (Don't Repeat Yourself) dan membuat aplikasi lebih mudah dipelihara.

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Inheritance memungkinkan class anak mewarisi properti dan method dari class induk dengan kata kunci `extends`.
2. Class induk `Controller` menampung kode yang dipakai bersama, yaitu `view()` untuk memuat View dan `redirect()` untuk berpindah halaman.
3. `MahasiswaController` yang mewarisi `Controller` menjadi lebih ringkas karena tidak menulis ulang `require_once` dan `header()`.
4. Visibilitas `protected` membuat method induk hanya dapat dipakai oleh class turunan sehingga encapsulation tetap terjaga.
5. Refactoring ini tidak mengubah perilaku aplikasi, tetapi membuat kode lebih rapi, mengurangi duplikasi, dan lebih mudah dikembangkan.

DAFTAR PUSTAKA

1. Politeknik Negeri Jember. 2026. Modul Praktikum Workshop SI Web Server (TIF330805), Acara 10.
2. The PHP Group. PHP Manual: Object Inheritance.
<https://www.php.net/manual/en/language.oop5.inheritance.php>
3. The PHP Group. PHP Manual: Visibility.
<https://www.php.net/manual/en/language.oop5.visibility.php>